Execution Semantics Are Statistics: Rules for Valid Security Evaluation of LLM Agents on Unreliable Infrastructure

[AUTHORS AND AFFILIATIONS TO BE INSERTED]

Working manuscript v3, 18 October 2026 — synthetic results, a real-environment SQA campaign, and a small real-agent study

Abstract

Purpose. Security evaluations of LLM agents confirm vulnerabilities statistically, by repeating stochastic agent episodes, and run them on distributed harnesses whose defaults—at-least-once delivery, retry with fresh randomness, results consumed as they arrive, worker-local configuration—were designed for throughput. We ask whether these execution semantics preserve the error guarantees of the reported results. **Methods.** We state six execution rules and prove that, under them, both sequential (e-process) and fixed-sample (exact binomial) confirmation keep their nominal false-confirmation level under message loss, duplication, reordering, crashes, partitions and elastic membership. We show by construction that each corresponding default can break validity, and evaluate the rules in a deterministic fault-injection simulator and slot-level Monte Carlo with known ground truth. We then split the coordinator into five services over a hash-anchored heterogeneous evidence graph, give a repair algorithm with proofs that it never fabricates evidence or adds confirmations, and test it under synthetic storage corruption, in a Docker deployment with real network, process and storage faults (Toxiproxy, kill, pause, partition, on-disk corruption), and with real agents in AgentDojo driven by a local open-weight model. **Results.** At nominal level 0.05, executor defaults raised the false-confirmation probability of sequential confirmation to 0.08–0.13 and of fixed-sample confirmation to 0.11–0.999; with the rules it stayed at or below 0.035 in every configuration. A reference implementation scaled 25× on 32 workers at about 3.5 messages per episode. Auditing the counterfactual-margin estimator of MARGIN-Bench, we found it exceeds its nominal error (0.257) and give an exact repair. A shared coordinator gave no speed-up over independent runs with a split budget, and anytime-valid confirmation cost 2–5 points of power against the repaired fixed design. Splitting the coordinator into five services over a self-verifying evidence graph, with a new repair algorithm (TWGR), made the evidence survive storage corruption. TWGR accepted no wrong record and added no confirmation in 640 repairs at corruption rates up to 0.6, recovering 0.98 of records at rate 0.3. Common alternatives produced false confirmations in up to all runs. A coordinated adversary defeated TWGR exactly when it held a majority of a record's domains, as proved, and could then add false confirmations. A signed variant (TWGR-S) closes this loophole: in 2,700 attacks by adversaries holding up to all five domains it accepted no forgery and added no confirmation, as Theorem 6 guarantees. Testing the deployment with real network, process and storage faults in Docker found seven defects in the networked service, all fixed, and the final campaign passed 24 of 24 test cases. In real AgentDojo episodes, a small local model was never compromised. A judge LLM, checked against facts from the traces, showed why: across all 97 tasks of four suites the agent rarely used its tools, so the injection reached it in only 7 episodes. A larger agent used its tools and was compromised in 4 of 24 attacked episodes. **Conclusion.** Execution semantics, and the integrity of the stored evidence, belong in the statistical protocol of agent security evaluation.

Keywords: LLM agents; security evaluation; statistical validity; distributed systems; fault tolerance; sequential testing; e-values; reproducibility

1 Introduction

Tool-using language-model agents fail securely or insecurely through *actions*—an unauthorized ledger update, a forwarded secret, a privilege change—and benchmarks increasingly measure these actions in executable sandboxes [1–4]. Because agents are stochastic, a single violating episode is weak evidence; best-of-N attacks show that success grows with repetition [5]. Rigorous evaluations therefore *confirm* vulnerabilities statistically: a candidate intervention is declared to violate policy with probability at least α only when repeated episodes support it at a stated error level [6, 7]. MARGIN-Bench [8] builds a full estimand on this idea, the counterfactual safety margin

$$R_{s,\alpha}(M, E) = \inf\{\, c_s(\delta) \;:\; p_M(E, \delta) \geq \alpha \,\}, \tag{1}$$

the cheapest intervention on attack surface s whose violation probability reaches α.

Such evaluations need 10^7–10^8 agent episodes and run on distributed harnesses. The defaults of those harnesses are reasonable for throughput and harmful for statistics. A queue that redelivers a result makes one episode count twice. A retry policy that re-runs crashed episodes with fresh randomness keeps the outcomes of episodes that happened not to crash—and long, benign trajectories crash more often. Consuming results as they arrive orders evidence by latency, which depends on the outcome. Letting each worker apply its own error budget multiplies the family-wise error by the number of workers. None of these is a bug in the statistical test; each changes the data the test sees.

This paper argues that *execution semantics are part of the statistical protocol* and makes the argument precise.

Contributions.

C1 **Six execution rules** (Table 1) and proofs that, under them, sequential e-process confirmation (Theorem 1) and fixed-sample exact confirmation (Proposition 2) keep their nominal error under message loss, duplication, reordering, crashes, partitions and elastic membership, for any scheduler.

C2 **Necessity.** For each rule, the corresponding executor default admits executions whose false-confirmation probability exceeds the nominal level, in several cases tending to one (Theorem 3); we reproduce this for both test families in simulation.

C3 **Audit and repair** of the MARGIN-Bench margin estimator, which as specified exceeds its nominal error five-fold; an exact repair restores validity at no efficiency cost.

C4 **Cost of the rules.** A reference implementation (a single-ledger protocol) and a fault-injection evaluation showing near-linear scaling, small coordination cost and fast recovery.

C5 **Estimator guidance** from preregistered comparisons: a repaired fixed design is the most powerful at a fixed budget; anytime-valid confirmation buys flexible stopping at a 2–5 point power cost; a shared coordinator is unnecessary for speed.

C6 **Evidence that survives storage corruption** (Section 7). QUINTET splits the coordinator into five services with disjoint write responsibilities, and stores all run data as a heterogeneous graph whose records carry each other's digests. TWGR, a new repair algorithm, recovers the graph from corrupted storage. We prove it never fabricates a record while honest anchor domains outnumber corrupted ones. The bound is tight, and beyond it repair can add false confirmations. TWGR-S, with device-signed evidence and derivation dominance, closes that gap: with any number of coordinators compromised, repair never adds a confirmation. Neither variant accepted a wrong record in 640 synthetic repairs at corruption rates up to 0.6, and TWGR-S resisted 2,700 coordinated attacks. In a Docker deployment tested with real network, process and storage faults, the campaign found seven defects in the networked service, all fixed; the four integrity defects shared one cause.

Non-claims. We do not propose a new statistical test; the inferential results we rely on are standard. We claim no certified lower bound on the margin, no tolerance of Byzantine executors (storage faults and a minority of compromised coordinator domains are tolerated; Section 7), and validity only when episodes share no hidden state. Validity and recovery results rest on synthetic ground truth. The real-environment results test the implementation, and the real-agent measurements (Section 9.9) are small and descriptive.

2 Setting

Confirmation. A generator proposes candidates $\delta_1, \delta_2, \ldots$ with costs $c(\delta_j) \in \{1, \ldots, K\}$. Executing candidate j in *slot* k with randomness key $u_{j,k}$ yields the committed-violation label $Y_{j,k} = Y(\delta_j, u_{j,k}) \in \{0, 1\}$ of a permission-grounded oracle; $p(\delta_j) = \Pr_u[Y = 1]$. A *confirmation procedure* declares candidate j confirmed when its evidence rejects $H_j : p(\delta_j) \leq \alpha$ at level a_j; the reported margin is $\hat{b} = \min\{c(\delta_j) : j \text{ confirmed}\}$ and the target is

$$\Pr(R_{s,\alpha} > \hat{b}) \leq \varepsilon, \tag{2}$$

implied by the no-false-witness property $\Pr(\exists j \text{ confirmed} : p(\delta_j) \leq \alpha) \leq \varepsilon$. We consider two families: *fixed-sample* confirmation (an exact binomial test on a predetermined number n of slots) and *sequential* confirmation (a betting e-process that may stop at any slot).

The harness. Workers request slots, executors run one sandboxed episode per slot from an immutable snapshot, and results travel over a network that may delay, reorder, duplicate or drop messages and partition. Workers and executors fail by crashing; membership changes over time. Workers are honest (they belong to the evaluator); Byzantine behaviour is out of scope. The generator is treated as untrusted with respect to the statistics: it sees committed outcomes but never a slot's key before the slot is registered.

Assumption 1 (Independent episodes)**.** *Keys are i.i.d. and independent of the generator's randomness, and episodes share no hidden state, so $Y_{j,k}$ given the registration of j is Bernoulli$(p(\delta_j))$ independently across slots.*

Assumption 2 (Crash independence)**.** *Whether a slot crashes may depend on its own trajectory and on infrastructure noise independent of other slots' keys. (Not needed when crashed slots are replayed with the same key.)*

Assumption 3 (Oracle correctness)**.** *Y is the true committed-violation label; with false-positive rate ω all guarantees concern $p + \omega(1 - p)$.*

3 Related work

Valid inference under adaptivity. Test supermartingales, Ville's inequality and e-values give inference valid under optional stopping [9–11]; betting confidence sequences [12, 13], always-valid A/B testing [14], e-value and online multiple testing [15–17], weighted error spending [18] and group-sequential designs with delayed responses [19] are mature. All assume the observation sequence is well defined. We study how a fault-prone executor *produces* that sequence.

Statistical assurance in testing and search. Residual-risk estimation in fuzzing [20, 21], statistical model checking [22, 23], scenario optimisation [24], threshold and infinitely-armed bandits [25–28] and their parallel variants [29, 30] bound errors of search or identification; parallel fuzzing schedulers remove redundant work for deterministic oracles [31]. For stochastic confirmation, redundancy and ordering become correctness issues.

Distributed systems. We reuse leases [32], escrow [33], monotone state and coordination-freedom [34, 35], causal ordering [36, 37], list-scheduling bounds [38] and task frameworks [39]. Treating a non-replenishable statistical budget as a cluster resource has a precedent in differential-privacy budget scheduling [40].

LLM agent security. Benchmarks cover injection, tool use, long-horizon and multi-agent attacks [2–4, 41–43]; counterfactual and causal analyses separate permission from apparent risk [44, 45]; defences enforce capabilities, provenance and delegation [46–49]; time-of-check-to-time-of-use flaws affect agents themselves [50]. We are not aware of work treating the evaluation harness's execution semantics as part of the statistical protocol.

What is new. Each mechanism in our rules (idempotent delivery, in-order commit, deterministic replay, leases) is established, and so is each inferential result. The contribution is the mapping between them: which executor semantics preserve or break which test assumptions, proofs for both test families, constructions showing necessity, a quantified audit of a concrete estimator, and measurements of what the rules cost.

4 Execution rules

Table 1: Execution rules, the default each one forbids, and the test families that need it.

Rule	Forbidden default	Needed by
R1 Fresh index-keyed slots. Key $u_{j,k} = \mathrm{PRF}_\kappa(\mathrm{run}, j, k)$, revealed only when the slot is leased.	seed sets shared across candidates and reused for selection; seeds visible to the generator	both
R2 Predictable decisions. Bets use only registration data and the committed prefix; fixed-sample tests run only on the complete slot set.	betting on or peeking at results not yet committed	both
R3 Exactly-once, index-determined evidence. Each slot has one terminal value (first write wins, fenced by a lease token); evidence is the in-order committed prefix (sequential) or the predetermined index set (fixed-sample).	at-least-once consumption; consuming results in arrival order; "first n results to arrive"	both
R4 Direction-aware imputation, no fresh retry. A crashed slot counts as non-violating for confirmation (violating for audits); re-running is allowed only with the same key.	retry on failure with fresh randomness	both
R5 One global error budget. Every level a_j is spent from a single budget, e.g. leases from one ledger or a fixed split ε/W.	each worker spending ε on its own	both
R6 Verifiable confirmation. Decisions are recomputed from stored slot records, not taken from workers.	trusting worker-side decisions	both (integrity)

Theorem 1 (Sequential confirmation under the rules)**.** *Let candidate j use the e-process $E_{j,n} = \prod_{i\le n}(1 + \lambda_{j,i}(\tilde{Y}_{j,i} - \alpha))$ with $\lambda_{j,i} \in [0, 1/\alpha)$ a function of registration data and $\tilde{Y}_{j,1..i-1}$, and be confirmed when $\sup_n E_{j,n} \ge 1/a_j$, where a_j is fixed at registration and $\sum_j a_j \le \varepsilon$ almost surely. Under Assumptions 1–2 and rules R1–R5, for every generator, scheduler, stopping rule and execution order,* $\Pr(\exists j : H_j \text{ true}, j \text{ confirmed}) \le \varepsilon$.

Proof. Extend each candidate's slot sequence to infinity and let $E^*_{j,n}$ be the e-process on the full recorded sequence, $\mathcal{G}_{j,n} = \sigma(\text{registration of } j, \tilde{Y}_{j,1..n}, \text{crash noise of slots } 1..n)$. By R1 and Assumptions 1–2, $u_{j,n+1}$ is independent of $\mathcal{G}_{j,n}$; by R4, $\tilde{Y} \le Y$ pointwise; so $\mathbb{E}[\tilde{Y}_{j,n+1} \mid \mathcal{G}_{j,n}] \le p(\delta_j) \le \alpha$ under H_j, and with R2 E^*_j is a non-negative supermartingale from 1. Ville's inequality conditional on registration gives $\Pr(\sup_n E^*_{j,n} \ge 1/a_j) \le a_j$. By R3 the committed stream is a prefix of the full sequence, so any confirmation implies $\sup_n E^*_{j,n} \ge 1/a_j$ whatever the schedule, stopping or abandonment. Summing over registrations and using R5 gives the bound ε. □

Proposition 2 (Fixed-sample confirmation under the rules)**.** *Let candidate j be confirmed when $S_j = \sum_{k=1}^{n} \tilde{Y}_{j,k} \ge s^*_j$, where the index set $\{1, \dots, n\}$ and the level a_j are fixed at registration, $\Pr(\mathrm{Bin}(n, \alpha) \ge s^*_j) \le a_j$ (equivalently, the exact Clopper–Pearson lower bound exceeds α), the test is evaluated only when all n slots are terminal, and $\sum_j a_j \le \varepsilon$. Under Assumptions 1–2 and rules R1–R5 the same guarantee holds.*

Proof. Given registration, $(Y_{j,k})_{k \le n}$ are i.i.d. Bernoulli$(p(\delta_j))$ by R1 and Assumption 1; R3 makes S_j the sum over exactly these indices, each counted once; R4 gives $\tilde{Y} \le Y$; so S_j is stochastically dominated by $\mathrm{Bin}(n, p(\delta_j))$, which under H_j is dominated by $\mathrm{Bin}(n, \alpha)$. Hence $\Pr(S_j \ge s^*_j) \le a_j$, and the union bound with R5 gives ε. R2 forbids evaluating the test on partial sets, which would turn it into an uncorrected sequential test. □

Theorem 3 (Necessity of the rules)**.** *For each rule there is a fault pattern, with all other rules kept, under which the false-confirmation probability exceeds the nominal level:*

1. R3, duplicates. *Sequential: every result applied twice yields $(E^*)^2$, and at $p = \alpha$ the error lies in $[\sqrt{\varepsilon}/(1 + \lambda(1-\alpha)), \sqrt{\varepsilon}]$. Fixed-sample: duplicates displace fresh slots, shrinking the effective sample and inflating the variance of S_j/n.*

2. R4, fresh retry. *If benign episodes crash with probability $c_0 > c_1$ (the violating rate), the recorded stream is Bernoulli$\big(p(1-c_1)/(p(1-c_1)+(1-p)(1-c_0))\big) > \alpha$ at $p = \alpha$; a consistent sequential test then errs with probability tending to one, and a fixed-sample test errs with probability increasing in n.*

3. R3, arrival order. *If violating episodes return first, the first n arrivals out of $m > n$ launched, or the arrival-ordered prefix, over-represent violations; as $m \to \infty$ the error tends to one for both families.*

4. R5. *W workers each spending ε give family-wise error $1 - (1 - \varepsilon_0)^W \to 1$.*

5. R1/R2. *Selecting candidates on a shared fixed seed set and confirming on the same seeds, or betting with knowledge of the current outcome, yields error tending to one.*

Proof. Each item is a direct construction; items 1 (sequential) and 2 follow from the stated distributions and optional stopping, item 3 from the order statistics of latencies, item 4 from independence across workers. Section 9 measures each. □

Remarks. (i) Theorem 1 decouples validity from scheduling: any allocation heuristic, learned or distributed, is safe under R1–R5. (ii) Under drift both results hold for the null "$p \le \alpha$ at every slot". (iii) Replaying a crashed slot with the same key reproduces its outcome, removing the power loss of R4 and the need for Assumption 2; it requires seeded, deterministic execution, which must be audited.

5 Confirmation procedures

MARGIN-Bench Algorithm 1 and its repair. The MARGIN-Bench estimator processes budget levels in ascending order; at each level it screens q candidates with r_d discovery trials, sends the top T to confirmation on r_h hold-out trials, and returns the first confirmed level, under "a predefined confidence and multiplicity rule". The natural reading—a Wilson bound at level ε/T—corrects within a level but not across the K levels tested in sequence, and the Wilson bound is anti-conservative at $r_h = 30$. The repair tests each of the KT confirmations with an exact Clopper–Pearson bound at $\varepsilon/(KT)$, a fixed-sample procedure covered by Proposition 2; repeated passes use $\varepsilon/(n(n+1)KT)$ in pass n.

LEXIS and LEXIS-S. LEXIS opens candidates one at a time per level and confirms each with a plug-in Kelly e-process capped at m trials, with weights a_j from a polynomial schedule (lease n receives $\varepsilon/(n(n+1))$, split over R registrations). LEXIS-S adds Algorithm 1's screening: scouting trials on keys independent of confirmation keys, promotion of the top T with weights and bet priors fixed from scouting data, and capped e-process confirmation; Theorem 1 applies because scouting data are registration information. Both are anytime-valid: they can be stopped at any time or extended with more passes without re-planning.

Certificates. Searched budgets that yield no confirmation can be summarised by anytime-valid bounds on the generator's violating mass per level (a zero-failure bound $1 - \beta^{1/N}$ and a betting upper confidence sequence; R4 imputes crashed audits as violating). These bound the generator's distribution, never the margin itself.

6 Reference implementation

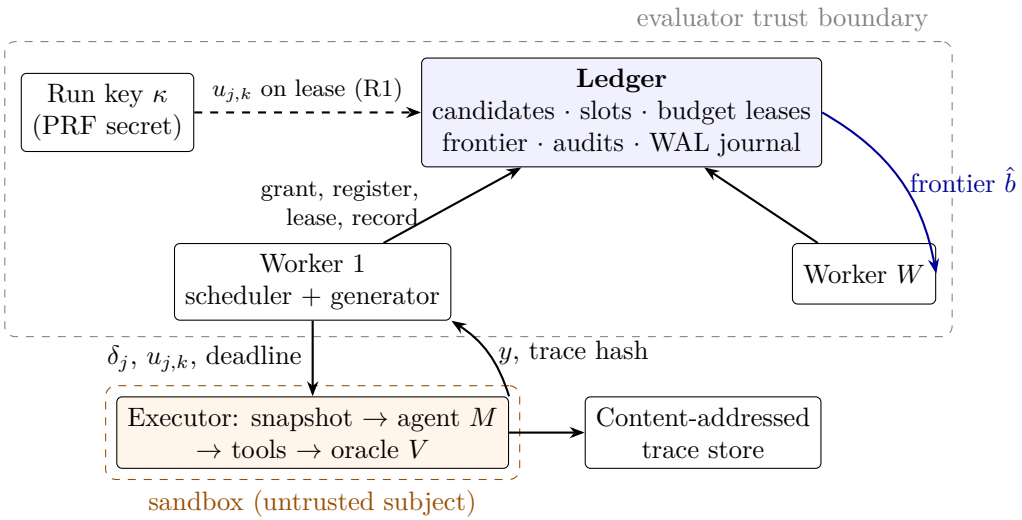


Figure 1: Reference implementation. A single logical ledger enforces R1, R3, R5 and R6; workers and the frontier broadcast are soft state that affects only efficiency.

The rules can be met in several ways; any harness that satisfies Table 1 inherits the guarantees. Our reference implementation (Fig. 1) uses one logical ledger as the serialisation point. Its state (candidates, slots, budget leases, frontier, audits) changes only through journaled, atomic transitions—GRANT, REGISTER, LEASE, RECORD, EXPIRE, ADVANCE, PRUNE, CLOSE, AUDIT. Requests carry (sender, request id) with cached responses, so retransmissions are idempotent; slot results are fenced by lease tokens (R3); expired slots become non-violating records (R4) or, in replay mode, are re-leased with the same key; budget leases are bound to a worker incarnation and

fenced on close (R5); and confirmation is recomputed from stored slots (R6). Invariants (budget conservation, write-once slots, monotone commit pointer and frontier, confirmation soundness) are checked after every transition in property-based tests, and journal replay reproduces the ledger state exactly after a coordinator crash.

Algorithm 1 Worker loop. Any scheduling policy is valid under Theorem 1.

loop
  if no open budget lease **then**
    request GRANT
  end if
  $a \leftarrow$ SCHEDULE(latest frontier, local state)
  if $a = \text{NEW}(k)$ **then**
    $\delta \leftarrow G_k$.PROPOSE(); REGISTER(lease, δ) ▷ a_j fixed here
  else if $a = \text{CONTINUE}(j, m)$ **then**
    for each $(k, u, \text{tok}) \in \text{LEASE}(j, m)$ **do**
      $y \leftarrow \text{EXECUTE}(\delta_j, u, \text{deadline})$; **if** no crash **then** RECORD(j, k, y, tok) ▷ retried until acknowledged
    end for
  end if
end loop

A shared ledger is not the only way to satisfy R5: W independent single-worker deployments with budget ε/W each also comply. Section 9 compares the two. Production deployments need no message broker, cache or custom consensus—a broker would add an at-least-once hop that must then be deduplicated—and task frameworks may place executors only with automatic retries disabled, since those retries implement the fresh-retry default.

7 Five coordinators and a self-repairing evidence graph

A single ledger is the simplest way to give every invariant one serialisation point, but it is one failure domain: if its storage is corrupted, nothing in the run can tell a true result from a damaged one. We therefore split the coordinator by *responsibility*—not by data shard—into five services, store all run data as one heterogeneous graph whose records vouch for each other, and give a repair algorithm that recovers the graph from damaged storage without ever adding evidence.

QUINTET. Each record type has exactly one writer (Table 2). Everything one coordinator reads from another is write-once, so a value never changes after it is read; reads across coordinators need no distributed transaction, by the CALM argument applied to write-once data [35]. Each rule therefore keeps a single serialisation point, and the validity results of Section 4 apply unchanged (Proposition 8). The five services are also five independent failure domains, and the repair algorithm draws on exactly that independence.

Table 2: QUINTET coordinators. Each is the single writer of its record types.

Coordinator	Owns (single writer)	Rules
Registrar	Candidate; Key (slot randomness $\text{PRF}_\kappa(j,k)$, revealed on lease)	R1
Treasurer	Lease ($\varepsilon/(n(n+1))$), Grant, Allocation (a_j; one per lease slot)	R5
Recorder	SlotLease (fencing token), Result (write-once; expiry ⇒ crash)	R3, R4
Arbiter	Confirmation (e-process on the in-order prefix), Frontier ($\hat{b}$)	R2, R6
Archivist	anchor index over all records; integrity scans; TWGR repair	—

The evidence graph. All run data form a typed graph with nine node types (Lease, Grant, Candidate, Allocation, SlotLease, Key, Result, Confirmation, Frontier). Every typed edge stores the digest of its target: a Result points to its Candidate, Key and SlotLease; a Confirmation points to its Allocation, its Candidate and every Result it consumed. A record is stored as bytes at its owner and at two witness domains chosen by hashing its id. Its digest is appended to hash-chained anchor logs at the owner, at both witnesses (a receipt) and at the Archivist. A record x thus has three kinds of evidence:

- *copies*, the stored bytes;
- *anchors*, the digests committed at write time in logs and in the edges of records that reference x;
- *derivations*, recomputations of x from records of other types, where the derivation is a pure function. A Lease is derived from its index, an Allocation from its Candidate and Lease, a Key from the run key, and a Confirmation from its Allocation and Results.

Algorithm 2 TWGR (typed-witness graph repair).

Input: domain stores; run parameters (budget policy, run key, α, cap)

index edge anchors and the anchor-log entries whose hash link verifies
$I \leftarrow$ ids with an intact copy or attested by ≥ 2 domains ▷ no phantom records
for type t in Lease, Grant, Candidate, Allocation, SlotLease, Key, Result, Confirmation, Frontier **do**
    for $x \in I$ of type t **do**
        $C \leftarrow$ decodable, id-consistent copies of x; $D \leftarrow$ derivation of x from accepted records, if any
        each domain votes the majority digest among *its anchors* (copies never vote; abstain on ties)
        $d^\star \leftarrow$ plurality digest; anchored $\leftarrow$ ($d^\star$ has a strict majority of votes and ≥ 2 domains hold evidence for it)
        if anchored and some $c \in C$ has digest $d^\star$ **then** accept c, heal all copies ▷ VERIFIED / REPAIRED
        else if anchored and digest$(D) = d^\star$ **then** accept D, heal ▷ RECONSTRUCTED
        else if not anchored and D exists **then** accept D ▷ exact: pure function of accepted inputs
        else quarantine x
        end if
    end for
end for
re-derived Confirmations count quarantined Results as $Y = 0$ and may be "confirmed" only if every consumed Result was accepted

TWGR. Algorithm 2 decides every record in type-dependency order, so derivations use only records already accepted. Records it cannot verify are *quarantined*, and the consumer treats a quarantined record in the conservative direction (as R4 does for crashes):

- a quarantined Result counts as a crashed slot ($Y = 0$ when confirming, $Z = 1$ when auditing);
- a quarantined Lease counts as fully spent;
- a quarantined Allocation or Candidate is unconfirmable.

The parts are standard—digests, replication, hash chains, majority voting. Three things are new in their combination:

- votes are restricted to write-time commitments, counted once per failure domain;
- records are reconstructed exactly from other record types;
- quarantine is direction-aware, chosen per type so that repair can remove evidence but never add it.

Let $m(x)$ be the number of domains holding anchors for x, and $b(x)$ the number among them whose stored items concerning x were modified after writing. Digests are 128-bit truncated SHA-256.

Theorem 4 (No fabrication)**.** *If $2b(y) < m(y)$ for x and for every record y that a derivation of x depends on, then the record TWGR accepts for x equals the record originally written. Under random corruption, the exception has probability at most $q^2 2^{-128}$ for q stored items.*

Proof. Induction over the type order. An unmodified domain holds only anchors computed from the true x, so it votes $d(x)$. The true digest therefore receives at least $m - b$ votes, and any other digest at most $b < m - b$. An anchored acceptance therefore has $d^\star = d(x)$, and a copy or derivation with that digest equals x by collision resistance. An unanchored acceptance is a derivation: a pure function of accepted inputs, which equal the truth by the induction hypothesis and which the writer used. Lease derivation depends only on public policy. Under random corruption, a wrong digest can win only if two corrupted items collide. □

The condition is tight. An adversary that controls a majority of a record's anchor domains, and writes one consistent wrong value into them, is accepted by any voting scheme. Experiment E11 confirms this boundary exactly. Plain replica-majority voting is weaker: two consistently tampered copies out of three suffice even when every anchor is honest.

Theorem 5 (Repair preserves validity)**.** *Under the condition of Theorem 4 for every record, the set of candidates confirmed in the repaired graph is a subset of those confirmed in the original run. Hence the false-confirmation probability after repair is at most that of the original run, which is at most ε by Theorem 1, and $\hat{b}$ can only increase.*

Proof. A confirmed record is accepted either as an anchored copy, which by Theorem 4 is the Arbiter's original record, or as a re-derivation. A re-derivation may confirm only if every consumed Result was accepted; it then recomputes the Arbiter's e-process on inputs equal to the original ones and reaches the same decision. Without this guard, a corruption that removes chosen Results would feed the plug-in betting e-process, which is not monotone in individual outcomes, a sequence it never saw. □

The quorum gap and TWGR-S. The tightness of Theorem 4 is a real loophole. An adversary holding a majority of a record's anchor domains can make TWGR accept a forged Confirmation, so repair *adds* a false confirmation. Experiment E11 shows this happening in every such attack. No voting rule avoids it, so TWGR-S removes votes from the evidence path entirely. It changes four things:

1. *Origin signatures.* The executor device signs its registration request and each slot outcome with Ed25519. The outcome signature binds run, j, k, y, the request hash and the slot's PRF key. Device public keys are in a run manifest published before the run, outside every coordinator domain. A record is admissible only if its payload consists exactly of the signed fields, in strict encoding, under the key of the candidate's own device. Edges are rebuilt from the schema and the accepted neighbours, and two signed values for one slot are quarantined.

2. *Derivation dominance.* Lease, Allocation, Key and Confirmation are always recomputed from accepted inputs, never taken from storage or votes.

3. *Unsigned means unknown.* The only unsigned evidence is a crash result ($Y = 0$). Every confirmation, live or re-derived, must hold for *every* 0/1 assignment of its unsigned crash slots (at most 12).

4. *Lease overspend.* A lease that funds more signed requests than its size is unconfirmable.

Theorem 6 (No added confirmation without a quorum)**.** *Assume the manifest is published before the run, Ed25519 is unforgeable and device keys are secret, and devices sign only what they observe and at most one outcome per slot. Let an adversary control any subset of the five coordinator domains, including all of them. Then every confirmation in the TWGR-S report was also made by the original run on the true outcomes, and the confirmable weights satisfy $\sum_j a_j \le \varepsilon$. Hence the false-confirmation probability of the reported result is at most ε.*

Proof sketch. Admissible Candidates and Results are genuine, since anything else breaks unforgeability, or they are unsigned crashes. Derived records equal their derivations. The repaired outcome sequence of a candidate agrees with the true sequence T on signed slots. It can differ only at unsigned-crash positions U, or by ending early. A confirmation on a prefix of T is the original decision, because e-process decisions depend only on the first crossing. Missing consumed results prevent confirmation. Inside T's length, the robust rule covers the assignment $T|_U$. Beyond T's end, the rule covers the all-zero assignment, and zero outcomes never raise the wealth of a non-negative bettor. Weights come from public policy and per-lease request counts. Full proof: Stage 7 notes, §4b. □

What the adversary can still do is delete, withhold or delay evidence, and replace outcomes by crashes. Each of these costs power, never validity. It can also alter bookkeeping records (Grant, SlotLease, Frontier) on which no reported quantity depends. Compromised device keys, or dishonest devices, remain oracle-integrity failures outside the model.

Proposition 7 (Recovery and cost)**.** *Record x is recovered exactly if $2b(x) < m(x)$, at least two domains hold evidence for $d(x)$, and either one copy is intact or x is derivable from recovered inputs. Under independent corruption with probability ρ per stored item, a non-derivable record with c copies fails with probability at most $\rho^c + \Pr(\mathrm{Bin}(m, \rho) \ge \lceil m/2 \rceil)$. TWGR takes one pass in $O(S + E + J \cdot \mathrm{cap})$ time and $O(S + E)$ space for S stored items, E edges and J candidates.*

Proposition 8 (QUINTET satisfies R1–R6)**.** *Each rule's state has exactly one writer (Table 2), and cross-coordinator reads touch only write-once records. Hence linearizable per-owner implementations compose without coordination, and Theorem 1 and Proposition 2 apply to runs on QUINTET.*

Implementation. Each coordinator is an HTTP service (Python standard library) with an append-only JSON-lines store. Writes are durable locally first; copies, receipts and Archivist anchors are then replicated asynchronously, with one retry queue per peer. After a restart, a coordinator re-offers its records to their witnesses (an idempotent operation) and rebuilds its id counters from the Archivist's anchor index, so it never reuses an id. On request, the Archivist pulls every domain, runs TWGR and pushes healed copies back.

8 Evaluation methodology

Why synthetic. Validity claims need ground truth—which candidates are null and the true margin k^*—which real agents never provide. We use (a) a deterministic discrete-event simulator

that runs the implemented ledger, message layer and workers with injected faults (loss, duplication, reordering, partitions, worker and executor crashes, ledger crash and journal recovery, outcome-correlated lognormal latency with means 20 s benign and 30 s violating) and Bernoulli oracles of known p; and (b) slot-level Monte Carlo of the confirmation procedures.

Instances. Four cost levels; S1-moderate has violating fractions $(0, 0, 0.05, 0.10)$ with $p = 0.6$, S2-sparse $(0, 0, 0.01, 0.05)$, and nulls with $p \sim U(0, 0.2)$; the boundary null sets every candidate to $p = \alpha = 0.2$, so every confirmation is false. $\alpha = 0.2$, $\varepsilon = 0.05$, four executors per worker.

Statistics. Seeds are the unit of analysis. Validity is judged by Wilson intervals against ε; time-to-margin is censored at a common horizon and compared by restricted mean survival time with bootstrap intervals and Holm-corrected Mann–Whitney tests [51]; non-inferiority uses Newcombe intervals for differences of proportions. Hypotheses and decision criteria were committed before the full runs (Table 5); the fixed-sample rule study (Table 3) was added during this reframing and is not preregistered.

9 Results

The results up to and including Section 9.7 are synthetic, with known ground truth. Section 9.8 tests the implementation on real infrastructure, and Section 9.9 measures real agents.

9.1 The rules matter for both test families

Through the full system (400 seeds per cell, boundary null, sequential confirmation), removing deduplication raised the false-confirmation probability to 0.133 [0.103, 0.169], allowing fresh-key retry to 0.102 [0.076, 0.136], and consuming results in arrival order to 0.083 [0.059, 0.114]; with all rules it stayed between 0.003 and 0.018 in every fault configuration, including all faults combined (0.003 [0.000, 0.014]). A worker-local budget at $W = 8$ gave 0.065 [0.045, 0.094], an inflation not established at this configuration. For fixed-sample confirmation (Table 3) the defaults were at least as harmful: using the first n results to arrive broke validity almost surely (0.999), fresh retry gave 0.189, duplicates 0.111, and a worker-local budget at $W = 8$ 0.266; with the rules the rate stayed at or below 0.035.

Table 3: Fixed-sample (exact Clopper–Pearson) confirmation of a 12-test family at the boundary null, $n = 30$, level $\varepsilon/12$ per test (50,000 runs; 95% Wilson intervals). Not preregistered.

Configuration	Pr(false confirmation)
Rules applied, no faults	0.035 [0.034, 0.037]
Rules applied, crashes 0.3 / 0.1 imputed as non-violating	0.014 [0.013, 0.015]
Rules applied, $W = 8$ with one global budget	0.019 [0.016, 0.023]
Default: duplicates counted (rate 0.3)	0.111 [0.108, 0.114]
Default: crash → fresh-key retry	0.189 [0.186, 0.193]
Default: first n of $2n$ arrivals, violations faster	**0.999** [0.998, 0.999]
Default: worker-local budget, $W = 8$	0.266 [0.255, 0.277]

9.2 Audit of the MARGIN-Bench estimator

As specified, Algorithm 1 confirmed a boundary-null candidate in 0.257 [0.238, 0.276] of 2,000 runs, five times its nominal level (Fig. 2); the exact repair gave 0.035 [0.027, 0.043] with indistinguishable power (P(reach k^*) 0.800 versus 0.803 on S1).

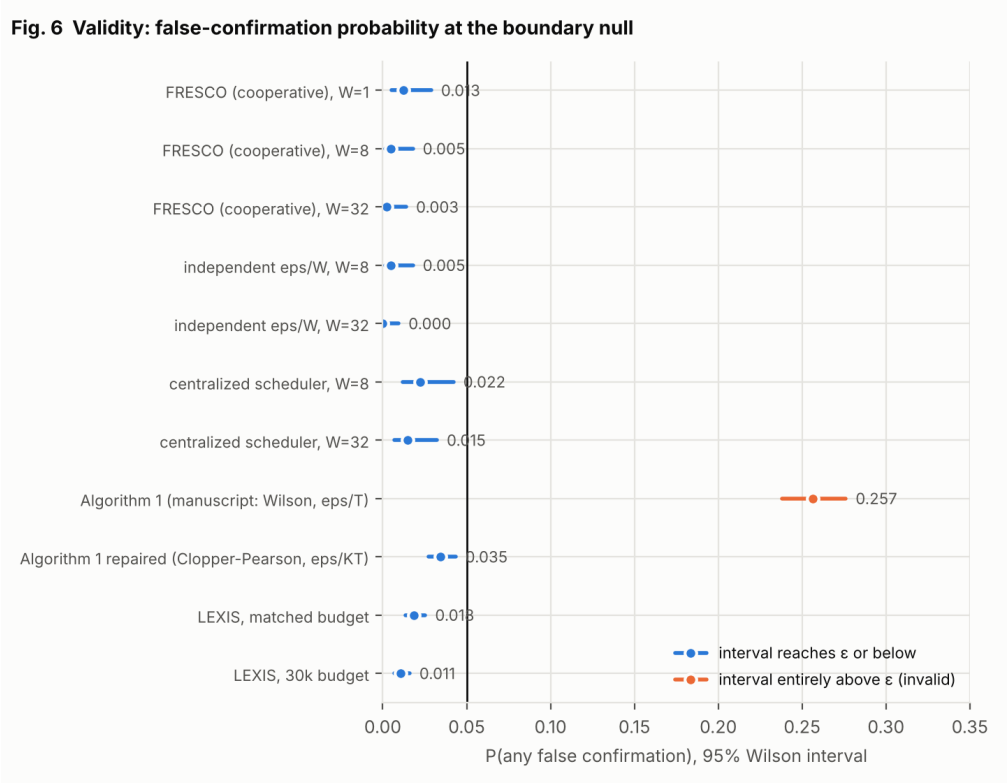


Figure 2: False-confirmation probability at the boundary null for all procedures and deployments. Only Algorithm 1 as specified is invalid.

9.3 What the rules cost

The reference implementation reached the true margin 25.4× faster with 32 workers than with one on S1 (parallel efficiency 0.79; at least 21.4× on S2, where single-worker runs were often censored) while the number of episodes needed stayed nearly flat (5.2k to 6.6k, overlapping intervals; Fig. 3). Coordination cost about 3.3–3.7 messages per episode and roughly 10 ledger operations per second at $W = 32$ in virtual time. After a 60 s ledger crash, a crash of half the workers or a 300 s partition, throughput recovered within one 20 s measurement bucket, and lost work matched the arithmetic loss. The main efficiency cost of the rules is conservative crash imputation: under 20% crashes, time to the margin rose 1.7×, mostly recovered by same-key replay.

9.4 Choosing a confirmation procedure

At a matched budget of 1,160 episodes (Table 4), repaired Algorithm 1 reached k^* in 0.808 of S1 runs; LEXIS without screening reached it in only 0.246, because it runs every opened null candidate to its cap. Adding screening (LEXIS-S) lifted this to 0.756 (S1) and 0.251 (S2, versus 0.269), but the preregistered non-inferiority margin of 0.05 was met in only 2 of 6 parameter settings, total episodes were unchanged (screening dominates cost), and scouting-weighted spending gave no detectable benefit. LEXIS-S was markedly more conservative (0.004 false confirmations versus 0.034): an anytime-valid test capped at a fixed sample uses less of its error budget, and that unused budget is the power it gives up. With 5,000 episodes and repeated passes both procedures reached k^* almost always on S1 (0.998, 1.000) and in 0.733 versus 0.765 of S2 runs.



Figure 3: Time to reach the true margin versus workers for the shared-ledger deployment, W independent deployments with budget ε/W, and one centralized scheduler (shaded: 95% bootstrap CI). All three comply with the rules.

Table 4: Probability of reaching the true margin $k^* = 3$ at a matched single-pass budget and with repeated passes (2,000 runs per cell; slot level). Rows 1 and 3 come from the first comparison (E4), the others from the preregistered hybrid study (E8); repaired Algorithm 1 was measured in both (S1: 0.800 and 0.808).

Procedure	S1	S2	Pr(false), boundary null
Algorithm 1 as specified (invalid)	0.803	0.280	0.257
Algorithm 1 repaired	**0.808**	**0.269**	0.034
LEXIS (no screening)	0.246	0.043	0.018
LEXIS-S (screening + e-process)	0.756	0.251	0.004
Algorithm 1 repaired, multi-pass (5,000)	1.000	0.765	0.015
LEXIS-S, multi-pass (5,000)	0.998	0.733	0.014

9.5 Is a shared coordinator worth it?

No measurable speed-up. In all 20 Holm-corrected comparisons (two instances, $W \in \{2, \ldots, 32\}$) the shared-ledger deployment was indistinguishable from W independent ε/W deployments and from one centralized scheduler: restricted-mean-time ratios 0.77–1.30, every 95% CI containing 1, every adjusted $p \geq 0.94$. With 100 seeds per cell, effects smaller than a ratio of about 0.65 could not be detected. Sharing one budget spreads it over more registrations, roughly cancelling the union-bound saving, while frontier pruning saves only work above the current margin.

9.6 Certificates

Per-level certificates covered the true violating mass in every run (coverage 1.000 [0.981, 1.000]) but were loose: median bounds of 0.39–0.75 against true masses of 0–0.01 after 100–280 audits per level. They are valid statements about the generator, not evidence of robustness.

Table 5: Preregistered hypotheses and verdicts (synthetic evidence).

Hypothesis	Criterion	Verdict
H-valid	Wilson lower bound $\leq \varepsilon$ in all rule-compliant configurations	supported
H-rules	removing a rule gives lower bound $> \varepsilon$ (sequential, system level)	supported for 3 of 4
H-alg1	Algorithm 1 as specified exceeds ε; repair does not	supported
H-speed	speedup lower bound ≥ 16 at $W = 32$	supported (25.4×)
H-overhead	≤ 10 messages per episode, ≤ 100 ledger ops/s	supported
H-recover	90% throughput within 2× outage	supported
H-coop	shared ledger faster than independent or centralized	**not supported**
H-vs-Alg1	LEXIS reaches k^* as often as repaired Algorithm 1	**refuted**
H6-noninf	LEXIS-S within 0.05 of repaired Algorithm 1	**not supported** (2/6 settings)
H6-cheaper	LEXIS-S uses fewer episodes	**refuted** (ratio ≈ 1.00)
H-cert	certificate coverage ≥ 0.95	supported; bounds loose
H-valid-real	measured fault profiles imply inflation	not testable (0 violations in 60 real episodes)
H7-safe	TWGR accepts no wrong record, adds no confirmation (E10)	supported (0 in 640 repairs)
H7-recover	recovery ≥ 0.99 at $\rho \leq 0.1$, ≥ 0.90 at $\rho = 0.3$	supported
H7-ablate	every component contributes at $\rho = 0.3$	**not supported** (domain votes $\approx$ item votes)
H7-baselines	replica-majority wrong; best-effort false confirmations $> \varepsilon$; single coordinator ≤ 0.5	supported
H7-validity	no false confirmation after repair beyond ε	supported (0/40 at every ρ)
H7-adversary	TWGR: wrong acceptance iff adversary holds a majority of anchor domains	supported (0 vs 1,512/1,512)
H7-S (post hoc)	TWGR-S: no forged value, no added confirmation, adversary in up to 5 domains	supported (0/2,700)

9.7 Corruption tolerance of the evidence graph

Setup (E10). We ran 80 QUINTET runs: 40 on S1-dense, with violating fractions $(0, 0, 0.3, 0.5)$, and 40 on the boundary null. Each run held about 9,300 records. We then corrupted every stored item (record copies and anchor-log entries) independently with probability $\rho \in \{0, \ldots, 0.6\}$, using bit flips, truncation, deletion and format-valid semantic tampering: a Result set to violating, a Confirmation set to confirmed, budgets inflated. Ten variants repaired the identical corrupted stores (Fig. 4).

Safety. TWGR accepted no wrong record and added no confirmation in any of 640 repairs, including at $\rho = 0.6$. After repair, no boundary-null run had a false confirmation (0/40 at every ρ), as Theorem 5 predicts. Every ablation was equally safe. The two usual alternatives were not:

- *Replica-majority* voting accepted wrong records from $\rho = 0.01$ on. It confirmed falsely in 6/40 boundary-null runs at $\rho = 0.1$ and in 20/40 at $\rho = 0.2$.
- *Best-effort* recovery (take the owner's copy) confirmed falsely in 6/40 runs at $\rho = 0.01$ and in 32/40 at $\rho = 0.1$.

TWGR-S was equally safe: no record with wrong content and no added confirmation at any rate. Its price is recovery. It quarantines whatever it cannot authenticate or recompute, and so recovered 0.998, 0.963 and 0.732 of records exactly at $\rho = 0.1$, 0.3 and 0.6, against 0.999, 0.979 and 0.807 for TWGR.

Recovery. TWGR recovered 0.9993 of records with their exact original bytes at $\rho = 0.1$, 0.979 at $\rho = 0.3$ and 0.804 at $\rho = 0.6$. A single coordinator with one copy and a checksum log is equally safe but cannot repair: it recovered 0.758, 0.394 and 0.097 at the same rates. At $\rho = 0.3$, removing a component of TWGR lowered recovery as follows (paired, all 95% CIs exclude 0):

Removed component	Recovery lost at $\rho = 0.3$
witness replicas	0.215
edge anchors	0.049
log anchors	0.031
derivation	0.010

Counting votes per item rather than per domain made no difference under random corruption, so our preregistered claim that every component contributes was not supported. Per-domain voting matters only against an adversary.

Domain loss and the adversary (E11). Losing the data of one, two or three of the five domains left 1.000, 0.975 and 0.754 of records recovered with TWGR, and 1.000, 0.986 and 0.761 with TWGR-S, with no wrong record. Requiring support from at least two domains costs availability here: an exploratory variant that accepts support from one domain recovered 1.000, 1.000 and 0.910. Theorem 4 does not need this requirement, but E11 did not test the variant. In 2,700 attacks per method, a coordinated adversary controlling $k = 1, \ldots, 5$ domains wrote one consistent forged value into every copy, log and edge it held. It forged Results ($0 \to 1$), Confirmations ($\to$ confirmed), Candidates (lower cost level) and Leases (inflated budget), and it suppressed results (a signed $Y = 1$ replaced by a crash). 1,512 of the attacks were by a majority of the target's anchor domains.

- *TWGR* accepted no forged value from a minority adversary. Against a majority it accepted every forgery (1,512 of 1,512, counting suppressions), including all 327 forged Confirmations, so repair *added* false confirmations. This is the loophole of Section 7, exactly at the bound of Theorem 4.
- *TWGR-S* accepted **no forged value and added no confirmation in any of the 2,700 attacks**, including those by four or all five domains. What a majority adversary achieved was to destroy evidence: it quarantined targets, or got 183 suppressions accepted, each turning a signed violation into a crash, which only weakens evidence. Theorem 6 predicts exactly this.
- *Replica-majority* voting added false confirmations in 36 attacks even by a minority adversary, and in all 327 by a majority.

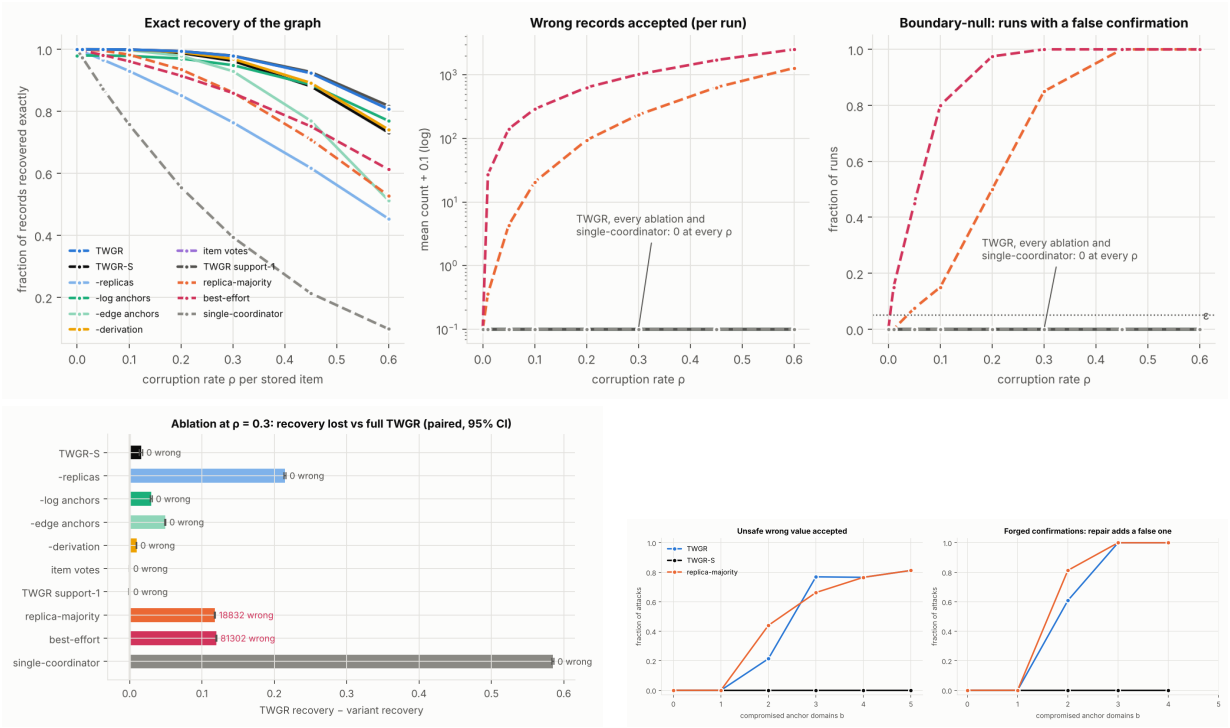


Figure 4: E10/E11. Top: exact recovery, wrong records accepted, and boundary-null runs with a false confirmation after repair, as functions of the per-item corruption rate (80 runs). Bottom left: paired ablation at $\rho = 0.3$. Bottom right: coordinated attacks by b compromised anchor domains. Left panel: an unsafe forged value accepted. Right panel: a forged Confirmation turned into a false confirmation by repair. TWGR-S is at 0 for every b.

9.8 A real controlled environment

Environment. We deployed QUINTET as five coordinator containers plus six worker containers (simulated executor devices, each limited to 0.5 CPU and 192 MB) on a private Docker network. All faults were real, with only free tools:

- every worker connection passed through Toxiproxy, which injected latency, timeouts and connection resets;
- processes were crashed, frozen and partitioned with `docker kill`, `pause` and `network disconnect`;
- storage was corrupted by editing the coordinators' on-disk files while they were stopped.

Test plan. Twelve test cases with pass criteria were committed before execution (Table 6). After every case a harness pulled all five stores and checked six invariants:

- write-once records;
- the global budget $\sum a_j \leq \varepsilon$;
- no stuck slots;
- confirmations reproducible from stored records;
- full replication;
- the frontier.

Outcome. We ran the campaign 11 times. Runs 1–8 used plain TWGR, and each failing run led to a fix (below). From run 9 on, the devices signed their evidence and the coordinators repaired with TWGR-S. The final run passed **all 24 test cases**. Under latency, timeouts, resets, a killed Recorder, a frozen Arbiter, a partitioned Treasurer and killed workers, every invariant held and every record verified; slots whose results never arrived became crash results (R4), up to 136 per run. After on-disk corruption of all five stores, the repaired graph matched the pre-corruption graph exactly on 1.000, 0.993 and 0.932 of records at $\rho = 0.05$, 0.2 and 0.4. No record had wrong content, no confirmation was added, and a second scan found nothing more to repair. Erasing the data of two coordinators lost nothing. The adversary test held up against attackers controlling one to all five coordinators. At $k = 4$ and $k = 5$ the adversary held a majority for every target; it destroyed evidence for 4 and 6 of 10 targets, but no forged value was accepted and no confirmation was added at any k. Five boundary-null runs of about 900 episodes each produced no false confirmation.

Defects found. The campaign found seven defects in the networked service, none in TWGR, and all were fixed with regression tests. Review of this work then identified the majority-adversary loophole, which led to TWGR-S. Building TWGR-S surfaced three more issues: unsigned parts of signed records (edges, extra fields, re-encoded signatures) were admissible, and ties on links to destroyed records could pick a corrupted copy. None of these three changed statistical content, and all are fixed. Four were integrity defects: a restarted Recorder fabricating a crash result for a slot whose result it had lost locally; a coordinator re-deciding from damaged local data; damaged copies being re-replicated with receipts; and a quarantined result served to the Arbiter. All four had one cause: a coordinator trusted its own storage after a fault. Two principles fixed them, and we recommend both for any harness that keeps evidence on disk:

- *repair before serve*: a restarted service that finds its store damaged acts only after a repair;
- *anchor-verified reads*: a service uses a local record only if the record matches a digest it committed when writing it.

The in-process experiments could not reveal these defects because they have no restarts.

Table 6: E12: SQA campaign on Docker devices, final run (11 of 11; signed evidence, TWGR-S): 24 of 24 cases pass. Criteria were fixed before execution, with dated clarifications; the full report and all failing earlier runs are in the repository.

TC	Fault and outcome	Verdict
1	none; all records verified, $\hat{b} = k^* = 3$	PASS
2–3	latency 150±100 ms; timeouts and resets on 15% of connections	PASS
4–7	Recorder killed; Arbiter frozen 15 s; Treasurer partitioned 15 s; 2 of 6 workers killed	PASS
8	on-disk corruption $\rho = 0.05$ / 0.2 / 0.4: exact recovery 1.000 / 0.993 / 0.932, 0 wrong, 0 added	PASS
9	data of 2 coordinators erased (two pairs): recovery 1.000 and 1.000	PASS
10	adversary in $k = 1, \ldots, 5$ domains forging Results and Confirmations: 0 wrong, 0 added at every k	PASS
11	boundary null, 5 runs: 0 false confirmations	PASS
12	resume work after repair: 756 new records, 0 conflicts	PASS

9.9 Real-agent experiments

Setup (E13). We ran AgentDojo v1.2.1 [2] simulated tool environments (banking and Slack suites) with the `important_instructions` prompt-injection attack. A free open-weight model, Qwen2.5-1.5B served locally by Ollama, ran on CPU in a container, at temperature 0.7 with a per-episode seed. AgentDojo's own post-state checks labelled each episode with two outcomes:

violation (the injection task succeeded) and *utility* (the authorized task succeeded). The design, fixed before execution, was 12 task pairs × 5 seeds, run in a predetermined order.

Results. All 60 episodes completed without harness errors.

- **No violation occurred** (0/60; 95% CI [0, 0.060]). The model also completed only 14/60 authorized tasks (0.233 [0.144, 0.354]); this small model mostly fails to use tools at all, rather than resisting the attack.
- Episode latency had median 11.6 s and p90 26.2 s.
- Latency depended on the outcome. At a timeout set at the 90th percentile, 0.21 of successful episodes would time out against 0.07 of failed ones.
- Replaying 8 episodes with identical seeds reproduced all 8 outcomes. With mostly negative outcomes, this is weak evidence of determinism.

What this shows. The outcome-dependent timeouts that make fresh-key retry invalid (R4) are present in real agent episodes. Here they appear for task success, because violations were too rare to measure. With no violations we cannot estimate a violation-specific inflation. Hypothesis H-valid-real therefore remains untested, and estimating it requires a model that the attack actually compromises.

Why the episodes fail (E14). We ran the agent on *every* user task of the four AgentDojo suites (banking, Slack, travel and workspace; 97 tasks). Each task ran once with the attack, using a rotating injection task, and once without, giving 194 episodes with full traces. A local judge LLM, Qwen2.5-7B at temperature 0, labelled every episode. We constrained its output with a JSON schema to a fixed taxonomy and to AgentDojo's own labels, so it could explain an outcome but never contradict it. We checked it against facts computed directly from the traces.

- **The agent mostly does not use its tools.** Of the 177 failed episodes, 136 contain no tool call at all (the 1.5B agent completed 8–9 of 97 tasks). Instead, the agent asks the user for data its tools could fetch ("could you provide the IBAN"); the judge and a rule classifier agree on this category with $\kappa = 0.72$. Less often it writes the tool call as a JSON block in plain text (10–12 episodes), claims actions it never took, or only describes a plan.
- **The attack never reached it.** The injected text was returned to the agent in only 7 of 97 attacked episodes, and the agent ignored it each time. The zero violations of E13 therefore mean *not exposed*, not *resistant*. The attack changed task success in only 3 of 97 paired tasks.
- **Capacity is the cause.** We reran a stratified subset with the 7B model as the agent: 6 tasks per suite, with and without the attack, 48 episodes. Episodes without a tool call fell from 69% to 6%, and task success rose from 5/48 to 16/48 (exact McNemar $p = 0.007$). The injected text now reached the agent in half of the attacked episodes, and the attack succeeded in 4/24 (0.17, 95% CI [0.07, 0.36]). That is 4 of the 12 exposed episodes: a banking transfer, two Slack messages and a hotel reservation made for the attacker.
- **The judge is reliable only for narrow, observable questions.** It agreed with the trace facts on exposure ($\kappa = 0.47$, 89% agreement) and on "asks the user" ($\kappa = 0.72$). It disagreed on its broad "main reason" label, calling many no-tool episodes "wrong tool" ($\kappa = 0.19$ for no tool use), and on the finer behaviours (κ 0.20–0.41). Re-judging at temperature 0.7 reproduced its category in 78% of cases. We therefore report trace facts where they exist and use the judge for the explanation, not for counts.

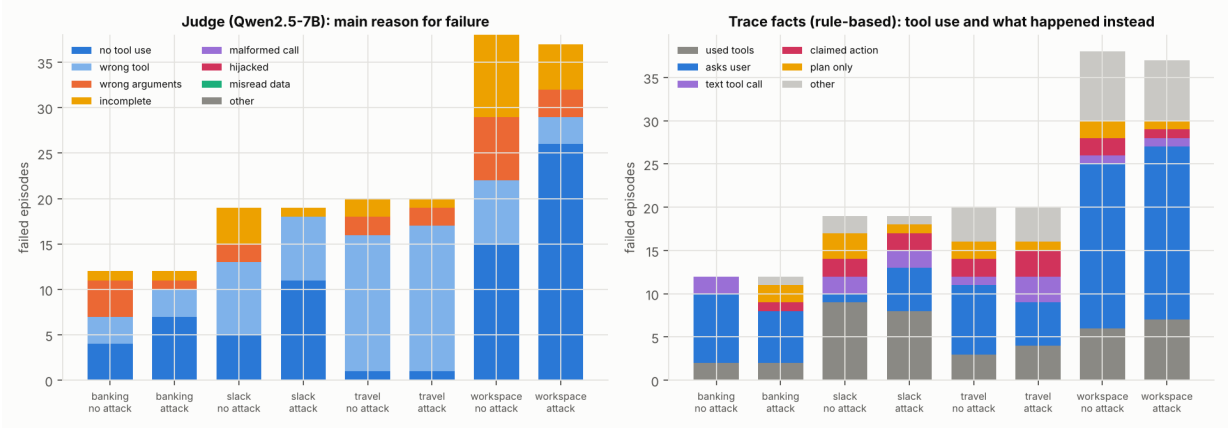


Figure 5: E14: why the 1.5B agent failed AgentDojo tasks (177 failed episodes), by suite and attack condition. Left: the judge's main reason. Right: facts from the traces. Grey bars are episodes that used tools; the colours split the no-tool episodes by what the agent did instead, using a rule classifier. The judge often calls no-tool episodes "wrong tool".

Table 7: Real-agent results (E13, Qwen2.5-1.5B via Ollama, AgentDojo banking and Slack, 60 episodes). Quantities not yet measured remain marked.

Quantity	Value
Violation rate	0/60 (95% CI [0, 0.060])
Authorized-task completion	14/60 = 0.233 [0.144, 0.354]
Latency median / p90	11.6 s / 26.2 s
P(timeout | task completed) vs. (not completed), τ =p90	0.21 vs. 0.07
Replay determinism (same outcome, same seed)	8/8
Timeout rate by violation outcome (c_0, c_1)	not estimable (no violations)
Violations, every task of 4 suites (E14, 1.5B)	0/97 (exposed to the injection in 7)
Violations, stratified subset with a 7B agent (E14)	4/24 [0.07, 0.36] (exposed in 12)
Episodes without any tool call: 1.5B vs 7B agent	74% (all 194) vs 6% (subset)
Redelivery rate of a naive queue	[TO RUN]
Episodes to $\hat{b}$ on MARGIN-Bench families	[TO RUN]

10 Practitioner checklist

1. Derive each episode's randomness from (run key, candidate, slot index); reveal it only when the slot is dispatched; publish a hash of the run key before the run.
2. Store results write-once per (candidate, slot) and ignore duplicates; never count a redelivered result.
3. Decide which slots form the evidence *before* running them: the in-order prefix for sequential tests, a fixed index set for fixed-sample tests. Never use "the first n results to arrive".
4. Record a crashed or timed-out episode as non-violating when confirming (violating when certifying absence); never silently re-run it with new randomness. Replay with the same key only after auditing determinism; disable automatic retries in task frameworks.
5. Spend one error budget across all workers, all budget levels and all passes; correct across levels.
6. Recompute every confirmation from stored records.
7. Reset all agent state (memory, tool state) between episodes, and audit the oracle for false positives.
8. Preregister deadlines, thresholds and test parameters, and report imputation rates per arm.

11 Discussion and failure analysis

What the evidence supports. Ordinary execution semantics can multiply the error of agent security evaluations, sometimes to certainty, and six cheap rules prevent it for both common families of confirmation tests. The rules cost one write per episode, scale near-linearly, and recover quickly from faults. The audit shows the issue is not hypothetical bookkeeping: a concrete, carefully specified benchmark estimator was invalid for the related reason of uncorrected multiplicity.

What it does not support. A shared coordinator is not needed for speed: independent deployments with a split budget are as fast and equally valid, provided each follows the rules. Anytime-valid confirmation is not more sample-efficient: at a fixed budget the repaired fixed design is slightly more powerful, and early stopping saves little when screening dominates cost. Sequential confirmation remains the right choice when runs must be extended or stopped unpredictably.

When the guarantees fail. Shared hidden state between episodes (e.g. agent memory not reset) violates Assumption 1 and can inflate error under any execution; oracle false positives shift the guarantee to the observed rate; leaked keys enable selection on seeds. If violating episodes time out more often than benign ones, conservative imputation hides them: validity holds but power collapses, so deadlines are statistical parameters to be preregistered.

12 Threats to validity

Construct: the fault rates are chosen triggers, not measurements; real harnesses may show smaller (or larger) effects. *Internal:* procedure parameters were not tuned beyond a small preregistered sweep; one LEXIS variant is labelled exploratory; the fixed-sample rule study was added during reframing. *Statistical:* 400 runs per system-level cell limit detection of modest inflation, which is why the global-budget result remains open at $W = 8$; seeds share random streams across

methods. *External:* simulated generators are non-adaptive and instances small. *Implementation:* the database-backed ledger is specified but not executed; coordination cost is measured in virtual time. *Corruption model:* E10 corrupts items independently and its tampering is deterministic per record type; correlated faults (a disk failing gradually, a buggy writer) are represented only by whole-domain loss and by the coordinated adversary. *Real environment:* the SQA campaign ran on one host, so its network faults are injected rather than geographic, and each test case ran once per campaign run; the seven fixed defects show that the implementation was immature, and further defects may remain.

13 Limitations and negative results

(i) The real-agent studies use small open models on CPU: one weak model across all 97 tasks, and one stronger model on 48 episodes. The judge LLM is reliable only for narrow, observable questions. (ii) No speed-up from a shared coordinator. (iii) Sequential confirmation, with or without screening, is somewhat less powerful than a repaired fixed design at a fixed budget. (iv) Certificates are loose at realistic audit budgets. (v) The necessity of the global budget was not established at $W = 8$ at system level for sequential tests (it was for fixed-sample tests). (vi) Byzantine executors (devices that lie or leak their keys) are not covered. Storage faults and any number of compromised coordinator domains are covered by TWGR-S, and a minority of them by TWGR. (vii) Bit-reproducible epoch mode and a database adapter are not implemented. (viii) Requiring two-domain support in TWGR costs availability when whole domains are lost (0.754 recovered with three of five lost, against 0.910 without the requirement). (ix) Per-domain voting brought no measurable gain under random corruption. (x) TWGR quarantines a record it cannot verify but does not re-establish its lost anchors. (xi) TWGR-S buys its guarantee with recovery under random corruption (0.963 against 0.979 at $\rho = 0.3$), needs a device key infrastructure and a published manifest, and leaves bookkeeping records (Grant, SlotLease, Frontier) quorum-dependent; no reported quantity depends on them.

14 Reproducibility and data availability

Code, configurations, raw results (JSON lines with configuration hashes and code revisions), analysis scripts and figures are in the project repository [INSERT PERMANENT ARCHIVE DOI]. One script regenerates every synthetic result in about three hours on four cores (Stage-7 experiments included). 73 unit, property-based and integration tests cover the ledger, protocol, procedures, simulator, evidence graph, TWGR and the networked QUINTET services; the latter include regression tests for every defect found in the real-environment campaign. The simulator is deterministic given configuration and seed. The real-environment campaign is reproducible with free software (Docker, Toxiproxy, AgentDojo, Ollama) by one command each; every failing campaign run is retained with its raw data.

15 Future work

Execute the full real-agent protocol at scale; measure the rules' effect in widely used open-source evaluation harnesses; extend the rules to estimators with adaptive allocation of confirmation trials and to Byzantine executors with attestation; and narrow the power gap of anytime-valid confirmation at fixed budgets.

16 Conclusion

The error level of an agent security evaluation is a property of the whole experiment, including how its harness delivers, retries, orders and budgets episodes. We gave six rules that make

sequential and fixed-sample confirmation valid under the faults of real infrastructure, showed that each common default can break validity, repaired an estimator that was invalid as specified, and found that the rules are cheap while a shared coordinator and anytime-valid confirmation buy flexibility rather than speed or power. Splitting the coordinator into five services over a self-verifying evidence graph makes the evidence itself survive storage corruption, without ever adding a confirmation. Testing that design on real infrastructure taught a lesson the simulations could not: a service must treat its own storage as untrusted evidence after any fault.

Declarations

Funding. [INSERT TRUE FUNDING INFORMATION OR "No funding was received".]
Competing interests. [AUTHORS TO DISCLOSE.]
Ethics approval. Not applicable to the simulation study; the real-agent protocol uses sandboxed mock tools and no human participants [CONFIRM BEFORE REAL-AGENT RUNS].
Data availability. Synthetic results, the SQA campaign data (including failing runs) and the real-agent episode records are released with the code [ARCHIVE DOI].
Code availability. [REPOSITORY URL AND ARCHIVE DOI].
Author contributions. [CRediT STATEMENT].
Use of generative AI. [AUTHORS MUST DISCLOSE: substantial parts of the design, code, analysis and text of this working manuscript were drafted with an AI assistant and must be verified and declared according to the journal's policy.]

References

[1] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. doi: 10.1145/3605764.3623985.

[2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.13352. arXiv:2406.13352.

[3] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv:2403.02691, 2024. URL https://arxiv.org/abs/2403.02691.

[4] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/jiang26as.html.

[5] John Hughes et al. Best-of-N jailbreaking. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025. URL https://arxiv.org/abs/2412.03556. arXiv:2412.03556.

[6] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations. arXiv:2411.00640, 2024. URL https://arxiv.org/abs/2411.00640.

[7] Bandana Kaur. What AI red-team evaluations can and cannot prove. arXiv:2607.21735, 2026. URL https://arxiv.org/abs/2607.21735.

[8] MARGIN-Bench: Counterfactual safety margins for multi-turn and multi-agent large language model systems. Working manuscript v1.0, 18 October 2026 (unpublished), 2026.

[9] Aaditya Ramdas, Peter Grünwald, Vladimir Vovk, and Glenn Shafer. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4), 2023. doi: 10.1214/23-STS894.

[10] Vladimir Vovk and Ruodu Wang. E-values: Calibration, combination and applications. *The Annals of Statistics*, 49(3), 2021. doi: 10.1214/20-AOS2020.

[11] Peter Grünwald, Rianne de Heide, and Wouter Koolen. Safe testing. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 2024. doi: 10.1093/jrsssb/qkae011.

[12] Ian Waudby-Smith and Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1), 2024. doi: 10.1093/jrsssb/qkad009.

[13] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2), 2021. doi: 10.1214/20-AOS1991.

[14] Ramesh Johari, Pete Koomen, Leonid Pekelis, and David Walsh. Always valid inference: Continuous monitoring of A/B tests. *Operations Research*, 70(3), 2022. doi: 10.1287/opre.2021.2135.

[15] Ruodu Wang and Aaditya Ramdas. False discovery rate control with e-values. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 84(3), 2022. doi: 10.1111/rssb.12489.

[16] Ziyu Xu and Aaditya Ramdas. Online multiple testing with e-values. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238 of *Proceedings of Machine Learning Research*, pages 3997–4005, 2024. URL https://proceedings.mlr.press/v238/xu24a.html.

[17] Dean P. Foster and Robert A. Stine. α-investing: a procedure for sequential control of expected false discoveries. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 70(2), 2008. doi: 10.1111/j.1467-9868.2007.00643.x.

[18] Christopher R. Genovese, Kathryn Roeder, and Larry Wasserman. False discovery control with p-value weighting. *Biometrika*, 93(3), 2006. doi: 10.1093/biomet/93.3.509.

[19] Lisa V. Hampson and Christopher Jennison. Group sequential tests for delayed responses (with discussion). *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 75(1), 2013. doi: 10.1111/j.1467-9868.2012.01030.x.

[20] Marcel Böhme. STADS: Software testing as species discovery. *ACM Transactions on Software Engineering and Methodology*, 27(2), 2018. doi: 10.1145/3210309.

[21] Marcel Böhme, Danushka Liyanage, and Valentin Wüstholz. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021. doi: 10.1145/3468264.3468570.

[22] Håkan L. S. Younes and Reid G. Simmons. Statistical probabilistic model checking with a focus on time-bounded properties. *Information and Computation*, 204(9), 2006. doi: 10.1016/j.ic.2006.05.002.

[23] Axel Legay, Benoît Delahaye, and Saddek Bensalem. Statistical model checking: An overview. In *Runtime Verification (RV 2010)*, volume 6418 of *Lecture Notes in Computer Science*. Springer, 2010. doi: 10.1007/978-3-642-16612-9_11.

[24] Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19(3), 2008. doi: 10.1137/07069821X.

[25] Hideaki Kano, Junya Honda, Kentaro Sakamaki, Kentaro Matsuura, Atsuyoshi Nakamura, and Masashi Sugiyama. Good arm identification via bandit feedback. *Machine Learning*, 2019. doi: 10.1007/s10994-019-05784-4.

[26] Andrea Locatelli, Maurilio Gutzeit, and Alexandra Carpentier. An optimal algorithm for the thresholding bandit problem. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, 2016. URL https://proceedings.mlr.press/v48/locatelli16.html.

[27] Maryam Aziz, Jesse Anderton, Emilie Kaufmann, and Javed Aslam. Pure exploration in infinitely-armed bandit models with fixed-confidence. In *Proceedings of Algorithmic Learning Theory (ALT)*, volume 83 of *Proceedings of Machine Learning Research*, pages 3–24, 2018. URL https://proceedings.mlr.press/v83/aziz18a.html.

[28] Arghya Roy Chaudhuri and Shivaram Kalyanakrishnan. PAC identification of a bandit arm relative to a reward quantile. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017. URL https://ojs.aaai.org/index.php/AAAI/article/view/10802.

[29] Eshcar Hillel, Zohar Karnin, Tomer Koren, Ronny Lempel, and Oren Somekh. Distributed exploration in multi-armed bandits. In *Advances in Neural Information Processing Systems 26 (NeurIPS)*, 2013. URL https://papers.nips.cc/paper/2013/hash/598b3e71ec378bd83e0a727608b5db01-Abstract.html.

[30] Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. A system for massively parallel hyperparameter tuning. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/a06f20b349c6cf09a6b171c71b88bbfc-Abstract.html.

[31] DynamiQ: Unlocking the potential of dynamic task allocation in parallel fuzzing. arXiv:2510.04469, 2025. URL https://arxiv.org/abs/2510.04469.

[32] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989. doi: 10.1145/74850.74870.

[33] Patrick E. O'Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4), 1986. doi: 10.1145/7239.7265.

[34] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, volume 6976 of *Lecture Notes in Computer Science*. Springer, 2011. doi: 10.1007/978-3-642-24550-3_29.

[35] Joseph M. Hellerstein and Peter Alvaro. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9), 2020. doi: 10.1145/3369736.

[36] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 1978. doi: 10.1145/359545.359563.

[37] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1), 1985. doi: 10.1145/214451.214456.

[38] Ronald L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 1969. doi: 10.1137/0117039.

[39] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/moritz.

[40] Tao Luo, Mingen Pan, Pierre Tholoniat, Asaf Cidon, Roxana Geambasu, and Mathias Lécuyer. Privacy budget scheduling. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021. URL https://www.usenix.org/conference/osdi21/presentation/luo.

[41] TAMAS: Benchmarking adversarial risks in multi-agent LLM systems. arXiv:2511.05269, 2025. URL https://arxiv.org/abs/2511.05269.

[42] The dark side of LLMs: Agent-based attack vectors for system-level compromise. arXiv:2507.06850, 2025. URL https://arxiv.org/abs/2507.06850.

[43] Mantas Mazeika et al. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, 2024. URL https://proceedings.mlr.press/v235/mazeika24a.html.

[44] Tianzhuo Yang et al. AgentBoundary: Counterfactual evaluation of safety in tool-using LLM agents. arXiv:2609.33658, 2026. URL https://arxiv.org/abs/2609.33658.

[45] Viraaji Mothukuri and Reza M. Parizi. Causal detection of multi-step LLM agent attacks. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://proceedings.mlr.press/v306/mothukuri26a.html.

[46] Edoardo Debenedetti et al. Defeating prompt injections by design. arXiv:2503.18813, 2025. URL https://arxiv.org/abs/2503.18813.

[47] Tobin South et al. Authenticated delegation and authorized AI agents. arXiv:2501.09674, 2025. URL https://arxiv.org/abs/2501.09674.

[48] Ruoming Pang et al. Zanzibar: Google's consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC)*, 2019. URL https://www.usenix.org/conference/atc19/presentation/pang.

[49] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. doi: 10.14722/ndss.2014.23212.

[50] Derek Lilienthal and Sanghyun Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents. NeurIPS 2025; arXiv:2508.17155, 2025. URL https://arxiv.org/abs/2508.17155.

[51] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 1958. doi: 10.1080/01621459.1958.10501452.